

BLOC 5 — Systèmes et Réseaux

Dossier de soutien

M2-Shop · Observabilité sécurisée

Concevoir et mettre en œuvre l'architecture d'un SI

Référentiel	ANSSI — Guide d'hygiène informatique / Zone Trust Model
Environnement	3 VMs Debian 12 · Vagrant · Ansible (ansible_local)
Compétences	C5.1.1 à C5.4.1

La société M2-Shop (plateforme de commerce électronique) souhaite refondre complètement sa stratégie d'observabilité. Suite à un audit de sécurité récent, l'ANSSI a pointé du doigt la porosité de l'ancienne infrastructure de supervision. Ma mission en tant qu'expert en architecture des systèmes d'informations est de concevoir et déployer une plateforme d'observabilité centralisée et durcie, capable d'absorber les métriques et les logs de la zone de production tout en garantissant un cloisonnement strict des réseaux d'administration.

Le Poc montre une observabilité compatible Zero-Trust : 3 zones, 4 flux lac, recette rejouable et testé

Inventaire des composants

VM	Rôle	IP	Zone	Ressources
web-prod	Nginx, Node Exporter, Zabbix Agent2, Promtail, Webhook receiver	10.0.20.20	DMZ Production	1 vCPU, 1 Go RAM
fw-router	nftables Zero-Trust, routage inter-zones	10.0.20.1 / 10.0.10.1	Inter-zones	1 vCPU, 512 Mo RAM
supervision	Prometheus, Alertmanager, Zabbix Server, Loki, Grafana	10.0.10.5	DMZ Administration	2 vCPU, 2 Go RAM

1. Grille Bloc 5

Points de contrôle	Démonstration
C5.1.1 Existant	Avant = LAN plat. Après = 3 zones
C5.1.2 Contraintes	ANSSI, Windows, pas de backup (assume en lab)
C5.1.3 Cahier des charges	4 flux FORWARD + 2 plans
C5.2.1 Architecture	Schéma HLD + LLD (IPs, ports)
C5.2.2 Recette	Commande → attendu
C5.2.3 PoC	Démo 5–7 min
C5.3.1 Intégration	<code>ansible_local + --limit</code>
C5.3.2 Automatisation	Playbooks + <code>healer</code>
C5.3.3 Compte-rendu	Tableau du 7 sept.
C5.4.1 Guide	Grafana + interdits

2. Existant et contraintes (C5.1.1 / C5.1.2)

Avant

- Boutique sur un LAN plat, admin et prod mélangées.
- Pas de matrice de flux.
- Pas d’observabilité
- Panne Nginx = un humain en root.
- Reconstruction à la main.

Après (cible)

```
web-prod 10.0.20.20 ↔ fw-router .20.1 / .10.1 ↔ supervision 10.0.10.5
DMZ Prod          DROP + 4 flux                DMZ Admin
```

Aucun réseau commun prod/admin. Le SRE pilote depuis Windows (`vagrant ssh`, Grafana :3443).

Contraintes

- **ANSSI** : DROP, moindre privilège, preuve de refus (timeout 22).
- **Métier** : le site 80/443 reste public, **hors** le routeur.
- **RTO Nginx** : < 60 s → mesuré < **1 s**.
- **Technique** : pas d'Ansible sur Windows → `ansible_local` dans chaque VM.
- **Routes statiques** obligatoires (sinon le NAT VirtualBox, Zero-Trust inopérant).
- **Limites assumées** : HTTP sur 9095 et Loki ; Grafana `admin/admin` ; **pas de backup** (on reconstruit via `vagrant` en lab).

Workshop Cible POC

M2-Shop — Plateforme de commerce électronique

Contexte	Valeur
Référentiel	ANSSI — Guide d'hygiène informatique / Zone Trust Model
Environnement	3 VMs Vagrant + VirtualBox (Debian 12 Bookworm)
Zone Production	web-prod — 10.0.20.20 (DMZ Production)
Zone Administration	supervision — 10.0.10.5 (DMZ Admin / Supervision)
Pare-feu inter-zones	fw-router — 10.0.20.1 / 10.0.10.1 (nftables Zero-Trust)
Hyperviseur	VirtualBox + Vagrant + Ansible (<code>ansible_local</code>)
Référence sujet	Workshop Évalué M2 — Infrastructure Cloud

3. Architecture, HLD,LLD et matrice de flux (C5.1.3 / C5.2.1)

- **HLD (High Level Design)** : zones, acteurs, flux métier, frontières de confiance. Public cible : Jury / RSSI / Métier.
- **LLD (Low Level Design)** : IPs, ports, services, fichiers, unités systemd. Public cible : Intégrateur (administrateur système / réseau, ingénieur DevOps / Cloud, ESN ou prestataire externe).

Cette section décrit l'architecture réseau de la plateforme d'observabilité sécurisée déployée pour M2-Shop, conformément aux exigences ANSSI du sujet. L'ensemble de l'infrastructure est défini en Infrastructure as Code (IaC) via Vagrant et Ansible, permettant une reconstruction complète en moins d'une heure.

L'infrastructure est organisée en trois zones réseau physiquement cloisonnées, sans aucun réseau privé VirtualBox commun entre la zone Production et la zone Administration. Tout flux inter-zone transite obligatoirement par le pare-feu dédié fw-router.

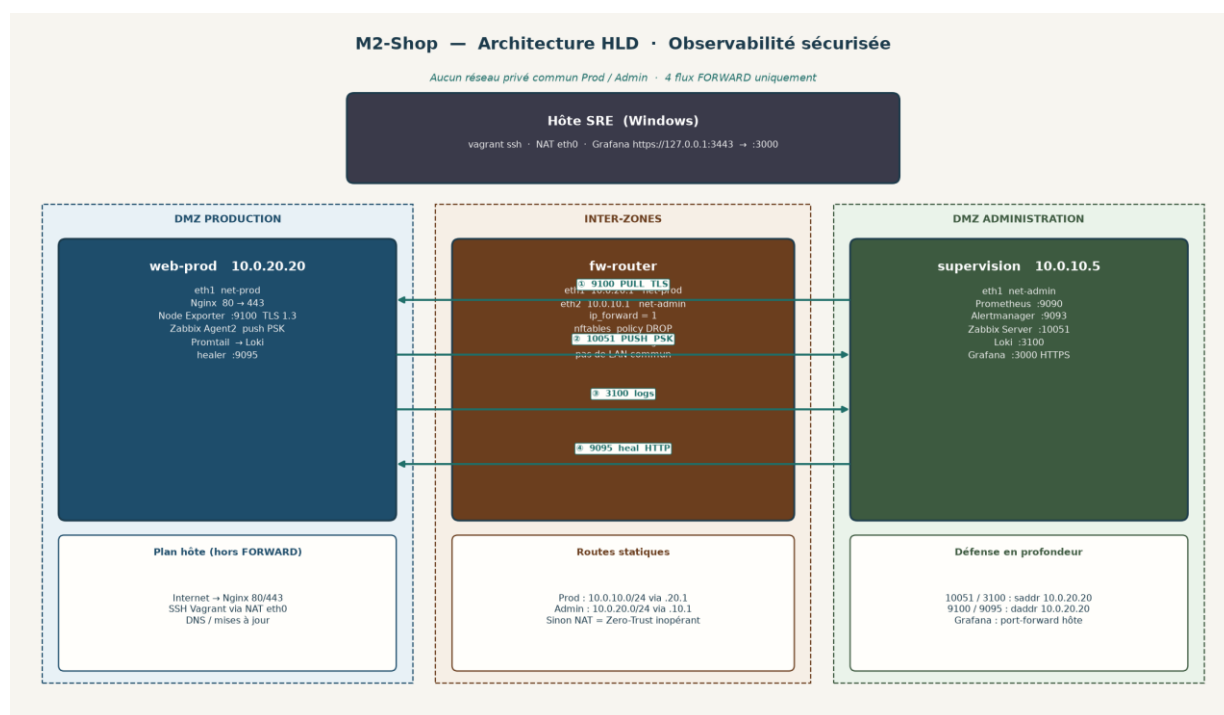


Schéma d'architecture M2-Shop — 3 zones cloisonnées

Figure — HLD : DMZ Production, pare-feu inter-zones, DMZ Administration. Les flèches numérotées sont les seuls flux FORWARD.

1. HLD — High Level Design

1.1 Objectif

Rendre une boutique web (M2-Shop) **observable** sans ouvrir la production à l'administration. La supervision est une zone de confiance distincte. Tout échange inter-zones est explicite, journalisé, et testé.

1.2 Acteurs

Acteur	Zone	Intention
Client Internet	Public	Consulter le site HTTPS
SRE	Hôte Windows	Piloter, voir Grafana, SSH Vagrant (NAT)
Services prod	DMZ Prod	Servir le site, pousser métriques/logs, s'auto-réparer
Plateforme admin	DMZ Admin	Collecter, alerter, dashboarder
Pare-feu	Inter-zones	Autoriser 4 flux, dropper le reste

1.3 Découpage en zones (modèle de confiance)

Zone	Confiance	Contenu
Internet	Non fiable	Clients du site
DMZ Production	Métier exposé	Nginx + collecteurs durcis
Inter-zones	Point de contrôle unique	fw-router (nftables DROP)
DMZ Admin	Supervision	Prometheus, Loki, Grafana, Zabbix, Alertmanager
Hôte SRE	Poste d'exploitation	Pas de playbook distant, pas de réseau commun prod/admin

Il n'existe **aucun** LAN partagé entre web-prod et supervision (`intnet net-prod` ≠ `net-admin`).

1.4 Flux métier (niveau HLD)

1. **Clients → Nginx** : HTTP redirigé vers HTTPS. N'emprunte **pas** fw-router.
2. **Prometheus → Node Exporter** : scrape PULL chiffré (Admin → Prod).
3. **Zabbix Agent → Zabbix Server** : push PSK (Prod → Admin). Le serveur n'initie jamais.
4. **Promtail → Loki** : centralisation des logs Nginx (Prod → Admin).
5. **Alertmanager → healer** : remédiation Nginx (Admin → Prod).
6. **SRE → Grafana** : HTTPS via port-forward `127.0.0.1:3443` (plan hôte).

1.5 Principes HLD

- Politique **deny by default** sur FORWARD et sur chaque hôte.
- **Défense en profondeur** : même si fw-router est contourné, Loki/Zabbix n'acceptent que `10.0.20.20`.
- **IaC** : l'architecture est le `Vagrantfile` + `ansible/site.yml`, pas un dessin orphelin.
- **PRA** : reconstruction, pas de restore (hors périmètre backup).

2. LLD — Low Level Design

2.1 Adressage et interfaces

Hôte	Interface	Réseau	IP
web-prod	eth0	NAT VirtualBox	10.0.2.15
web-prod	eth1	<code>net-prod</code>	10.0.20.20/24
fw-router	eth0	NAT	10.0.2.15

fw-router	eth1	net-prod	10.0.20.1/24
fw-router	eth2	net-admin	10.0.10.1/24
supervision	eth0	NAT	10.0.2.15
supervision	eth1	net-admin	10.0.10.5/24

Routes (systemd) :

- web-prod : 10.0.10.0/24 via 10.0.20.1 dev eth1
- supervision : 10.0.20.0/24 via 10.0.10.1 dev eth1

2.2 Matrice FORWARD (LLD)

Deux plans : **FORWARD inter-zones** (fw-router uniquement) et **INPUT/OUTPUT hôte** (SSH, ICMP, DNS, site public, Grafana). Ne pas les fusionner : Nginx 80/443 et SSH 22 n'apparaissent pas dans la chaîne FORWARD.

Plan inter-zones (FORWARD fw-router)

Zone Source	IP Source	Zone Dest.	IP Dest.	Protocole	Port	Chiffrement	Justification
DMZ Admin	10.0.10.5	DMZ Prod	10.0.20.20	TCP	9100	TLS v1.3 + Basic Auth	Scraping Prometheus (PULL)
DMZ Prod	10.0.20.20	DMZ Admin	10.0.10.5	TCP	10051	TLS-PSK 64 hex	Zabbix Agent mode Actif (Push)
DMZ Prod	10.0.20.20	DMZ Admin	10.0.10.5	TCP	3100	HTTP (réseau isolé)	Promtail -> Loki, logs Nginx
DMZ Admin	10.0.10.5	DMZ Prod	10.0.20.20	TCP	9095	HTTP en clair	Alertmanager -> webhook /alert (auto-healing)

Plan hôte (INPUT / OUTPUT locaux, hors FORWARD)

Source	Destination	Proto	Port	Chaîne	Justification
Hôte SRE / Vagrant (NAT eth0)	les 3 VMs	TCP	22	INPUT web-prod, fw-router, supervision	SSH labo. Interdit en FORWARD (timeout inter-zones).
Any (destiné à l'hôte)	VM locale	ICMP	echo	INPUT des 3 VMs	Diagnostic. Pas de FORWARD ICMP.
VM locale	Internet (NAT)	UDP/TCP	53	OUTPUT des 3 VMs	DNS (APT / GitHub).
web-prod, fw-router	Internet (NAT)	TCP	80/443	OUTPUT	Mises à jour paquets.
supervision	Internet	TCP	443	OUTPUT supervision	Webhook Slack #alertes-m2shop.

Internet / client	web-prod 10.0.20.20	TCP	80/443	INPUT web-prod uniquement	Nginx public. Absent de FORWARD fw-router.
Hôte Windows SRE	supervision	TCP	3000 (invité)	INPUT supervision	Grafana HTTPS dans la VM ; depuis Windows : 127.0.0.1:3443 → guest 3000.

2.3 Composants

Composant	Hôte	Version / écoute	Unité
Nginx	web-prod	1.30.5, 80 → 443, TLS auto-signé	nginx.service
Node Exporter	web-prod	1.8.2, :9100 TLS 1.3	node_exporter.service
Zabbix Agent2	web-prod	7.0, push	zabbix-agent2.service
Promtail	web-prod	3.5.1 → :3100	promtail.service
webhook-receiver	web-prod	:9095	webhook-receiver.service
nftables + ip_forward	fw-router	/etc/nftables.conf	nftables.service
Prometheus	supervision	2.55.1, :9090	prometheus.service
Alertmanager	supervision	0.27.0, :9093 localhost	alertmanager.service
Zabbix Server + MariaDB	supervision	7.0, :10051	zabbix-server
Loki	supervision	3.5.1, :3100	loki.service
Grafana	supervision	13.1.0, :3000 HTTPS	grafana.service

2.4 Fichiers et comptes clés

Élément	Chemin / compte
Playbook	ansible/site.yml (4 rôles)
nftables routeur	roles/fw-router/templates/nftables-router.conf.j2
nftables hôtes	nftables-webprod.conf.j2, nftables-supervision.conf.j2
Sudoers healer	/etc/sudoers.d/supervision — une commande
Script remédiation	/usr/local/bin/restart-nginx.sh
Logs healer	/var/log/healer/remediation.log
Vault	/etc/ansible-vault-pass.txt (hors /vagrant)

3. Traceabilité HLD → LLD → code

Décision HLD	Traduction LLD	Preuve
2 DMZ sans LAN commun	2 intnet Vagrant	Vagrantfile

Point de contrôle unique	fw-router FORWARD DROP + 4 accept	nftables-router.conf.j2
Zabbix ne « tire » pas	Agent <code>ServerActive</code> , pas de FORWARD 10050	zabbix_agent2.conf.j2
Collecteurs bornés	saddr 10.0.20.20 sur 10051/3100	nftables-supervision.conf.j2
Remédiation non-root	sudoers chirurgical	sudoers-supervision.j2
Rejouable	ansible_local --limit	site.yml

Schéma d'architecture M2-Shop — 3 zones cloisonnées

Qui fait quoi

VM	IP	Rôle
web-prod	10.0.20.20	Nginx, Node Exporter, Zabbix Agent, Promtail, healer
fw-router	.20.1 / .10.1	Seul routeur, nftables DROP
supervision	10.0.10.5	Prometheus, Loki, Grafana, Zabbix, Alertmanager

4 flux FORWARD seulement

Sens	Port	Usage	Chiffrement
Admin → Prod	9100	Prometheus scrape	TLS 1.3 + bcrypt
Prod → Admin	10051	Zabbix push	PSK
Prod → Admin	3100	Promtail → Loki	HTTP (labo)
Admin → Prod	9095	Auto-healing	HTTP (labo)

Deux plans

- **FORWARD** = uniquement le tableau ci-dessus.
- **Hôte** = SSH 22, DNS, site 80/443, Grafana 3000. Ça **ne traverse pas** fw-router.

4. Demonstration du poc (C5.2.2 / C5.3.3)

Un test = **commande + attendu + Résultat obtenu.**

Jalon	Test	Resultat.
A	ip -br a	eth1/eth2 = IPs du DAT
A	Routes	via .20.1 / via .10.1
A	SSH 22 inter-zones	TIMEOUT_OK
B	metrics sans auth	401
B	Prometheus	health=up
B	Zabbix	heartbeat working again 20:09
C	Loki	nginx_access + nginx_error
C	Grafana	dashboard M2-Shop + Explore
D	Préchecks	healer = 1 commande sudo ; services active
D	Crash Nginx	21:13:44 firing → 204 → relancé même seconde

.

5. Intégration et automatisation (C5.3.1 / C5.3.2)

Pourquoi `ansible\_local`

- Hôte Windows : pas d'orchestrateur.
- Vault hors partage `/vagrant` (sinon Ansible exécute le fichier mot de passe).
- `--limit` : chaque VM ne configure **qu'elle-même** (respecte le cloisonnement).

4 couches = `site.yml`

7. `common` — socle
8. `fw-router` — Jalon A
9. `web-prod` — prod
10. `supervision` — collecte / alertes

Automatisation

- Vagrant = les 3 VMs et les 2 `intnet`.
- Ansible = nftables, TLS, Grafana, Zabbix...
- systemd = routes + services après reboot.
- `healer` = **une** commande : `systemctl restart nginx.service`.

Pas automatisé: bump (monté de version) OpenSSL si bug PSK ; pas de job de sauvegarde.

6. Guide SRE (C5.4.1)

Usage quotidien (POC)

- Grafana : `https://127.0.0.1:3443` — `admin / admin` — dossier **M2-Shop**.
- Explore Loki : `{job="nginx_access"}` ou `{job="nginx_error"}`.
- Logs VM en **UTC** ; Grafana en heure Paris (UTC+2).

Interdits

Ne pas faire	Pourquoi
<code>curl https://10.0.20.20/</code> depuis Admin	443 n'est pas en FORWARD
SSH prod → admin (port 22)	DROP
<code>scp</code> vers 10.0.10.5	22 interdit
Hostname <code>fw-router</code>	Noms : <code>fw-router-prod</code> / <code>fw-router-admin</code>

Explication :

`curl https://10.0.20.20/` depuis Admin → « 443 n'est pas en FORWARD »

Depuis la VM supervision (DMZ Admin), il y aura une tentative de joindre le site Nginx de web-prod en HTTPS. Ça doit échouer (timeout), parce que le port **443 n'est autorisé qu'en INPUT sur web-prod** (accès public), et **jamais en FORWARD** à travers fw-router. Autrement dit, l'admin n'a aucune raison d'aller visiter la boutique par le routeur inter-zones : seuls les flux de supervision (9100, 9095) traversent. Si le curl répondait, ça voudrait dire qu'un flux 443 Admin→Prod existe alors qu'il n'est pas dans la matrice.

SSH prod → admin (port 22) → « DROP »

Depuis web-prod, tenter un SSH vers la supervision. La politique nftables par défaut est **DROP**, et le port 22 n'est pas dans les 4 flux FORWARD autorisés. La connexion doit donc être silencieusement jetée (timeout, pas de « connection refused »). C'est la preuve que la production ne peut pas « rebondir » vers la zone d'administration — protection contre le mouvement latéral.

`scp` vers 10.0.10.5 → « 22 interdit »

Même logique, formulée côté transfert de fichiers : `scp` s'appuie sur SSH (port 22). Comme le 22 ne traverse pas fw-router, impossible de copier un fichier de la prod vers l'admin par ce biais. C'est ce qui justifie, le choix de faire remonter les artefacts via le partage /vagrant plutôt que par un `scp` inter-zones.

En prod utilisé le modèle « push vers une zone neutre » (le plus aligné Zero-Trust).

La prod dépose le fichier dans un **stockage objet** (S3, ou un serveur auto-hébergé) situé dans une DMZ dédiée, jamais directement dans la zone d'admin. L'admin va ensuite le lire depuis ce bucket. Avantage : la prod n'ouvre jamais de session vers l'admin, elle écrit seulement dans un « sas ». Le flux reste sortant et borné à une API HTTPS/mTLS ou le modèle « pull via bastion » (le plus courant en entreprise).

On **inverse le sens** : ce n'est pas la prod qui pousse, c'est l'admin (ou un bastion) qui va **chercher** le fichier — donc une connexion **Admin → Prod**, l'inverse exact de ce qui est interdit dans ton tableau. Cette connexion passe par un **bastion nominatif** : chaque opérateur s'authentifie par sa propre clé, la session est journalisée, et surtout le compte côté prod est verrouillé par une *forced command* (`command="rsync --server ..."` dans le `authorized_keys`), sans shell interactif. C'est le `rsync/scp` « autorisé mais bridé » : une seule commande possible, un seul sens, tracé.

Je retiendrai le push vers une zone neutre parce qu'il élimine tout chemin réseau entre la prod et l'admin. Le pull via bastion avec forced command reste une alternative acceptable si l'on veut réutiliser de l'existant.

Admin SSH = ``vagrant ssh`` depuis Windows.

Incident - PCO (Plan de continuité des opérations (ou PCA) en 4 phrases

11. Confirmer via Grafana / Slack / Prometheus — pas un curl 443.
12. DROP FORWARD sur 10.0.20.20 (`vagrant ssh fw-router`).
13. Preuves dans ``/vagrant``.

Explication ligne 6 et 7 : Ces deux lignes sont la **procédure de confinement d'un incident**: que faire concrètement si web-prod est compromis ou se comporte mal. C'est la déclinaison opérationnelle de la ligne « PCO : isolation FORWARD + artefacts vers /vagrant » Deux étapes : d'abord **isoler**, ensuite **conserver les preuves**.

1. « DROP FORWARD sur 10.0.20.20 (vagrant ssh fw-router) »

C'est l'action d'**isolation réseau** de la machine suspecte (web-prod, 10.0.20.20).

Concrètement : Se connectes au routeur avec `vagrant ssh fw-router` — et non à web-prod, ce qui est important, j'y reviens — puis tu ajoutes une règle **DROP dans la chaîne FORWARD** de nftables qui vise 10.0.20.20. Résultat : plus aucun paquet ne transite entre web-prod et le reste (la zone admin), le temps de l'investigation. Cela coupe les 4 flux inter-zones autorisés (9100 Node Exporter métriques système, 10051Zabbix Server (trapper), 3100 Loki pour l'ingestion de logs, d'auto-remédiation9095 Webhook healer) d'un coup, sans toucher au reste de l'infra.

Le point clé, c'est la **où** la règle est posé. Il faut bloquer **sur fw-router**, le point de passage obligé, et pas sur web-prod elle-même. Deux raisons :

- Si web-prod est compromise, on ne fait **pas confiance** à ses propres règles de firewall (un attaquant a pu les modifier). Le routeur, lui, est dans une autre zone et reste sain — c'est un point de contrôle fiable.

14. Silence Alertmanager en **localhost** sur supervision.

Un *silence* est une fonctionnalité native d'Alertmanager : c'est une **règle temporaire qui empêche une alerte de partir en notification**, sans toucher à la détection elle-même. Prometheus continue d'évaluer ses règles et l'alerte peut toujours passer en état *firing* — mais Alertmanager la « retient » et **n'envoie ni webhook, ni Slack** tant que le silence est actif. On définit un silence avec des **matchers** (par ex. `alertname="NginxDown"`, `instance="10.0.20.20"`) et une **durée** (30 min, 2 h...), après quoi il expire tout seul.

À quoi ça sert concrètement ? Deux cas typiques :

- **Maintenance planifiée.** On veut volontairement arrêter/redémarrer Nginx sur web-prod. Sans silence, ça déclencherait NginxDown → **le webhook healer relancerait Nginx** (< 60 s) et on recevrais une alerte Slack pour rien. On poses donc un silence sur NginxDown avant l'intervention, pour **neutraliser l'auto-remédiation et le bruit** le temps de ta manip.

Backup

Pas de dump. Détruire une VM = perdre l'historique. On ``vagrant up``. Git = seule sauvegarde de l'état voulu.

Dans le lab, l'infrastructure est entièrement reproductible depuis Git, donc je ne sauvegarde pas les VMs. En production, je séparerais deux choses : l'état voulu, reconstruit depuis Git par l'IaC, et les données, couvertes par une vraie politique de sauvegarde.

Exemple de politique de sauvegarde en production :

Chiffre	Règle	Menace couverte
3	3 copies des données (l'original + 2 sauvegardes)	Perte ou corruption d'une copie
2	Sur 2 supports ou technologies différents	Défaillance d'un type de support
1	1 copie hors site	Sinistre physique (incendie, inondation, panne régionale)
1	1 copie immuable ou hors ligne (air-gap)	Ransomware, administrateur malveillant, suppression accidentelle
0	0 erreur lors des tests de restauration	Sauvegarde inutilisable découverte trop tard

3 : trois copies

Principe : les données de production plus **au moins deux sauvegardes** indépendantes.

Pourquoi : avec une seule sauvegarde, si elle est corrompue au moment où on en a besoin, nous n'avons plus rien. Deux sauvegardes indépendantes rendent très improbable la perte simultanée des trois copies.

À éviter : un RAID ou une réplication n'est **pas** une sauvegarde. Si tu supprimes une table, la suppression est répliquée instantanément sur le miroir. Le RAID et la réplication répondent à la disponibilité. La sauvegarde répond à la récupération. Nous avons besoin des deux : la réplication pour tenir face à une panne, et des sauvegardes versionnées et immuables pour revenir en arrière après une erreur humaine ou un ransomware.

2 : deux supports différents

Principe : les copies ne doivent pas reposer sur la même technologie, ni sur la même baie ou le même compte.

Exemples de combinaisons :

- un disque local et un stockage objet (S3, Azure Blob) ;
- un NAS et une bande LTO (format de stockage de données sur bande magnétique. **Utilisation** : Archivage à long terme et sauvegarde sécurisée (cold storage) pour les entreprises. **Avantages** : Grande capacité, coût réduit par téraoctet et excellente durabilité) ;
- des « managed disks » et un « Recovery Services Vault ».

Pourquoi : un firmware bogue, une baie défaillante ou un compte cloud bloqué ne doit pas emporter toutes les copies en même temps.

1 : une copie hors site

Principe : au moins une copie **géographiquement séparée** du site principal.

Exemples :

- Azure(ou autre cloud provider) : stockage **GRS** (géo-redondant) ou réplication vers une **autre région** (par exemple France Central → France South ou West Europe) ;
- on-premise : un second datacenter, ou un prestataire externe d'externalisation.

Pourquoi : un sinistre physique détruit tout ce qui se trouve au même endroit.

Point RGPD : vérifie que la région de destination reste dans l'UE si les données sont personnelles.

1 : une copie immuable ou hors ligne

C'est l'**ajout clé face aux ransomwares**. Les attaquants modernes cherchent d'abord les sauvegardes pour les chiffrer ou les supprimer, *avant* de déclencher l'attaque sur la production.

Deux façons d'y répondre :

Immuable (WORM : Write Once, Read Many)

La copie ne peut être ni modifiée ni supprimée pendant une durée définie, **même par un administrateur**.

- Azure : *immutability policy* sur un conteneur Blob (en mode **locked**), ou *immutable vault* sur Azure Backup, avec **soft delete** activé ;
- AWS : S3 **Object Lock** en mode *Compliance* ;
- on-premise : *hardened repository* Linux avec Veeam, ou une appliance avec verrouillage.

Hors ligne (air-gap)

La copie est **physiquement ou logiquement déconnectée** : bande éjectée et stockée dans un coffre, disque débranché, ou réseau d'isolement qui ne s'ouvre que pendant la fenêtre de sauvegarde.

Bonnes pratiques associées :

- comptes de sauvegarde **séparés** de l'annuaire principal (pas de compte AD du domaine de prod) ;
- **MFA** sur la console de sauvegarde ;
- suppression soumise à une validation multi-personnes (Azure : *Multi-User Authorization*).

0 : zéro erreur à la restauration

Principe : les sauvegardes sont **vérifiées automatiquement** et **testées par de vraies restaurations**, à intervalle régulier.

Ce que ça recouvre :

- **contrôle d'intégrité** à chaque sauvegarde (checksums, vérification du job) ;
- **tests de restauration réguliers**, par exemple une base restaurée chaque mois sur un environnement isolé, avec vérification que l'application démarre ;
- **mesure des temps réels** par rapport aux objectifs :
 - **RPO** (*Recovery Point Objective*) : combien de données on accepte de perdre. Avec un dump toutes les 24 h, le RPO est de 24 h ;
 - **RTO** (*Recovery Time Objective*) : combien de temps on accepte d'être indisponible ;
- **procédure de restauration documentée** (runbook) que quelqu'un d'autre que toi peut suivre.

Pourquoi : une sauvegarde qu'on n'a jamais restaurée peut être vide, incomplète, chiffrée avec une clé perdue, ou trop lente à restaurer. On le découvre toujours le jour de la crise.

On retrouve la logique du « push vers une zone neutre » : la prod **écrit** ses sauvegardes dans un stockage qu'elle ne peut **ni lire ni supprimer**.

7. Démo 5–7 min (C5.2.3)

Action	Preuves
<code>vagrant status + ip -br a</code> sur fw-router	3 zones, eth1 prod, eth2 admin
timeout 22 puis targets Prometheus	DROP vs flux 9100 up
Dashboard M2-Shop + Explore Loki	Métriques + logs, 2 datasources
Logs healer ou <code>stop nginx + 20 s</code>	21:13:44, une commande sudo
Ouvrir <code>ansible/site.yml</code>	4 plays, <code>ansible_local</code> , pas depuis Windows

Crash live (option A) :

```
vagrant ssh web-prod -c "sudo systemctl stop nginx"
Start-Sleep -Seconds 20
vagrant ssh web-prod -c "sudo systemctl is-active nginx"
```

Attendu : **active**.

```
vagrant status
vagrant ssh fw-router -c "ip -br a"
vagrant ssh web-prod -c "timeout 5 nc -zv 10.0.10.5 22 || echo TIMEOUT_OK"
vagrant ssh supervision -c "curl -s http://localhost:9090/api/v1/targets"
vagrant ssh web-prod -c "sudo tail -5 /var/log/healer/remediation.log"
```

Vue d'ensemble, ces 5 commandes vérifient dans l'ordre : les VM tournent (1) → le routeur cloisonne bien les 2 zones (2) → un flux interdit est effectivement bloqué (3) → un flux autorisé fonctionne, côté métriques (4) → l'auto-remédiation laisse bien ses preuves (5). **résultat attendu** : running / 2 IP / TIMEOUT_OK / target up / lignes de log présentes.

1. **vagrant status**

Ne rentre dans aucune VM : demande à Vagrant l'**état des 3 machines**. Tu dois voir web-prod, fw-router et supervision toutes en running. C'est le contrôle préalable : inutile de tester les flux si une VM est éteinte. Si l'une est poweroff, tu la relances avant d'aller plus loin.

2. **vagrant ssh fw-router -c "ip -br a"**

Sur le routeur, affiche les interfaces réseau en mode bref (`-br = brief`, `a = address`). On vérifie que fw-router a bien **une ip dans chaque zone** : une IP en 10.0.20.1 (côté prod) et une en 10.0.10.1 (côté admin), interfaces UP. C'est la preuve que le routeur est correctement **bi-domicilié** — sans ça, pas de cloisonnement possible.

3. **vagrant ssh web-prod -c "timeout 5 nc -zv 10.0.10.5 22 || echo TIMEOUT_OK"**

LA commande la plus intéressante — c'est un **test négatif**. Depuis web-prod, elle tente d'atteindre le port **22 (SSH)** de la supervision (10.0.10.5) :

- `nc -zv` = netcat en mode scan (`-z`, pas d'envoi de données) et verbeux
- (`-v`) : teste juste si le port répond.
- `timeout 5` = abandonne au bout de 5 s. Indispensable, car un paquet **jeté par DROP** ne renvoie rien → la commande resterait bloquée sans ce garde-fou.
- `|| echo TIMEOUT_OK` = « si la commande échoue, affiche `TIMEOUT_OK` ».

Le résultat **attendu est l'échec** : le SSH prod→admin doit expirer (DROP, port 22 hors FORWARD), donc on doit voir **TIMEOUT_OK**. Ici, « ça plante » = **le firewall fonctionne**. Si au contraire la connexion réussissait, ce serait une faille du cloisonnement.

4. vagrant ssh supervision -c "curl -s http://localhost:9090/api/v1/targets"

Sur la supervision, interroge l'API de **Prometheus** (port 9090) en localhost, endpoint `/api/v1/targets`. `-s` = mode silencieux (pas la barre de progression). Ça renvoie du JSON listant **toutes les cibles scrapées et leur santé** (`health: "up" / "down"`). On vérifie que Prometheus voit bien le Node Exporter de web-prod (9100) en `up` — donc que le flux de métriques Admin→Prod fonctionne.

5. vagrant ssh web-prod -c "sudo tail -5 /var/log/healer/remediation.log"

C'est la **preuve de l'auto-healing** : on doit y lire les traces des relances Nginx déclenchées par Alertmanager (avec horodatage) Ça valide le bout de la chaîne : détection → alerte → webhook → relance du service.

Grafana : <https://127.0.0.1:3443>